

Java Backend Architecture

Interview Series

Chapter 15.7

Resilience Patterns

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 15.7 – Resilience Patterns

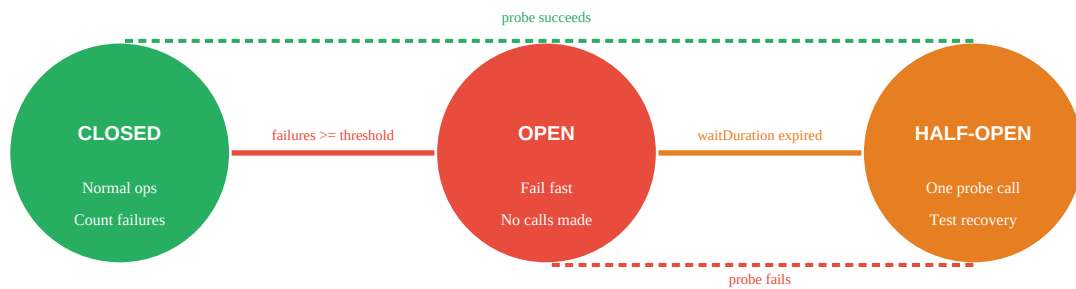
Introduction

In microservices, every network call can fail. Resilience patterns prevent cascading failures: a downstream service degradation should not take down your entire system. Resilience4j is the standard Java library for Circuit Breaker, Retry, Bulkhead, TimeLimiter, and RateLimiter. Combined with proper fallback strategies and chaos engineering, these patterns make distributed systems robust against the inevitable failures of networked dependencies.

Key Definitions

Circuit Breaker	Monitors failures to a downstream service. CLOSED: normal operation. OPEN: fail fast (no calls made). HALF-OPEN: test probe call. Prevents cascading failures by failing fast.
Retry	Automatically retry failed operations with configurable attempts, delay, backoff multiplier, and jitter. Handles transient failures (network hiccup). Must be idempotent.
Bulkhead	Isolates resources (thread pool or semaphore) per downstream service. Failure of one downstream cannot exhaust threads for others. Named after ship compartments.
Time Limiter (@TimeLimiter)	Sets a maximum execution time per call. If downstream doesn't respond within limit, throws TimeoutException. Combined with circuit breaker to count timeouts as failures.
Fallback	Alternative response when primary path fails: cached data, default value, alternative service, or graceful degradation. Last line of defense in resilience chain.
Rate Limiter	Limits the rate of calls to a downstream service. Prevents overwhelming a slower service. Can also protect your own API from being overloaded by clients.
Jitter	Random variation added to retry delay. Prevents the "thundering herd" problem where all retrying clients hit the service simultaneously after a shared failure.
Chaos Engineering	Intentionally injecting failures in production (or staging) to verify resilience. Netflix Chaos Monkey, Chaos Monkey for Spring Boot, LitmusChaos for Kubernetes.
Resilience4j	Lightweight Java resilience library. Functional/decorator approach. Works with Spring Boot via spring-cloud-starter-circuitbreaker-resilience4j. Integrates with Micrometer for metrics.
Sliding Window	Circuit Breaker counts failures in the last N calls (count-based) or last N seconds (time-based). Configurable. Time-based is better for services with variable load.

Circuit Breaker State Machine



Resilience4j: `@CircuitBreaker(name="paymentSvc", fallbackMethod="fallback")`. Metrics via Micrometer.

Bulkhead – Thread Pool Isolation

Service A – Single Thread Pool (No Bulkhead)

All downstream calls share pool: if Payment Svc hangs, all 50 threads exhausted, catalog calls also fail

Payment Svc
Thread Pool
(max 10)

Inventory Svc
Thread Pool
(max 20)

User Svc
Thread Pool
(max 5)

With Bulkhead: Payment Svc pool exhausted has ZERO impact on Inventory Svc or User Svc calls

Resilience Pattern Composition

Composition order: Rate Limiter wraps Retry, which wraps Circuit Breaker, which wraps Timeout
Outermost = Rate Limiter (reject before retry). Innermost = Timeout (per-attempt). Fallback is last resort.



If all patterns fail: Fallback returns cached data or default response — never propagates exception to client

Resilience4j: `@Retry + @CircuitBreaker + @TimeLimiter + @Bulkhead` composable as decorator chain

Circuit Breaker Configuration

Property	Default	Recommended Prod	Meaning
failureRateThreshold	50%	50%	Open circuit when failure rate >= this
slowCallRateThreshold	100%	80%	Open when slow calls >= this
slowCallDurationThreshold	60s	2s	Call duration considered "slow"
slidingWindowSize	100	20	Number of calls/seconds in window

waitDurationInOpenState	60s	30s	Time in OPEN before trying HALF-OPEN
permittedCallsInHalfOpenState	10	5	Probe calls in HALF-OPEN
minimumNumberOfCalls	100	10	Min calls before evaluating failure rate

Resilience4j – Spring Boot Circuit Breaker

```
# application.yml resilience4j: circuitbreaker: instances: paymentService:
failureRateThreshold: 50 slowCallRateThreshold: 80 slowCallDurationThreshold: 2s
slidingWindowSize: 20 waitDurationInOpenState: 30s permittedCallsInHalfOpenState: 5
minimumNumberOfCalls: 10 registerHealthIndicator: true // Java service @Service public class
PaymentClient { @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService") @TimeLimiter(name = "paymentService") @Bulkhead(name =
"paymentService", type = Bulkhead.Type.THREADPOOL) public CompletableFuture<PaymentResult>
processPayment(PaymentRequest req) { return CompletableFuture.supplyAsync(() ->
paymentGateway.charge(req)); } // Fallback – must have same signature + Throwable parameter
public CompletableFuture<PaymentResult> paymentFallback( PaymentRequest req, Throwable ex) {
log.warn("Payment circuit open, using fallback: {}", ex.getMessage()); return
CompletableFuture.completedFuture( PaymentResult.pending(req.getOrderId(), "Payment service
unavailable, will retry")); } }
```

Retry with Exponential Backoff + Jitter

```
# application.yml resilience4j: retry: instances: paymentService: maxAttempts: 3 waitDuration:
1s enableExponentialBackoff: true exponentialBackoffMultiplier: 2 enableRandomizedWait: true #
Jitter! randomizedWaitFactor: 0.5 # +/- 50% of delay retryExceptions: -
org.springframework.web.client.HttpServerErrorException - java.io.IOException
ignoreExceptions: - com.example.BusinessValidationException # Don't retry business errors //
Alternative: programmatic with @Retry annotation @Retry(name = "external-api", fallbackMethod =
"fallback") public String callExternalApi(String id) { return
restTemplate.getForObject("/api/data/" + id, String.class); } // Retry 1 after 1s, Retry 2
after 2s±random, Retry 3 after 4s±random
```

Bulkhead Configuration

```
# Thread Pool Bulkhead (async) resilience4j: thread-pool-bulkhead: instances: paymentService:
maxThreadPoolSize: 10 # Max 10 threads for payment calls coreThreadPoolSize: 5 queueCapacity:
20 # Queue up to 20 waiting calls keepAliveDuration: 20ms # Semaphore Bulkhead (sync)
resilience4j: bulkhead: instances: inventoryService: maxConcurrentCalls: 15 # Max 15 concurrent
calls maxWaitDuration: 100ms # Wait up to 100ms for permit // Usage - ThreadPool Bulkhead
requires CompletableFuture @Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
public CompletableFuture<PaymentResult> processPayment(PaymentRequest req) { return
CompletableFuture.supplyAsync(() -> paymentGateway.charge(req)); } // Semaphore Bulkhead works
with regular return types @Bulkhead(name = "inventoryService") public InventoryStatus
checkInventory(String productId) { return inventoryClient.getStatus(productId); }
```

Chaos Monkey for Spring Boot

```
<!-- pom.xml - Chaos Monkey for Spring Boot --> <dependency> <groupId>de.codecentric</groupId>
<artifactId>chaos-monkey-spring-boot</artifactId> <version>3.1.0</version> </dependency> #
application.yml chaos: monkey: enabled: true watcher: service: true # Inject chaos into
@Service beans rest-controller: true # Inject into @RestController assaults: level: 3 # 1-10:
probability of assault latency-active: true # Inject random latency latency-range-start: 1000 #
1-3 seconds latency-range-end: 3000 exceptions-active: true # Randomly throw exceptions
kill-application-active: false # Don't actually kill in dev # Chaos Monkey can be controlled
via Actuator API: # POST /actuator/chaosmonkey/assaults # { "latencyActive": true,
```

```
"latencyRangeStart": 2000, ... } // Use to verify your circuit breaker, retry, and fallback work correctly
```

Interview Q&A;

Q1. What are the three states of a circuit breaker?

CLOSED: normal operation, calls pass through. Counts failures. OPEN: circuit tripped, all calls fail fast immediately (no actual call made). Saves resources, prevents cascade. HALF-OPEN: after waitDuration expires, allows limited probe calls. If success: back to CLOSED. If failure: back to OPEN.

Q2. Why do we need a circuit breaker instead of just retrying?

Retries help with transient failures (brief network hiccup). But if downstream service is genuinely down or overloaded, retries make it worse—flooding a struggling service with more requests. Circuit breaker detects persistent failure and fails fast (returns error immediately without calling downstream), giving it time to recover.

Q3. What is the difference between count-based and time-based sliding windows?

Count-based: track failure rate over last N calls. Stable metric but biased by request rate. If calls are infrequent, a single failure = 100% failure rate. Time-based: track failure rate over last N seconds. Better for services with variable load. Resilience4j supports both: slidingWindowType=COUNT_BASED or TIME_BASED.

Q4. Why is jitter important in retry delays?

Without jitter: if 100 clients all fail simultaneously (server restart), they all retry at the same interval (1s, 2s, 4s...). This creates synchronized thundering herd — the server gets hit by 100 clients exactly at retry time. With jitter: each client adds random variation to delay, spreading retries across time → server recovers gradually.

Q5. What is the difference between Semaphore and ThreadPool Bulkhead?

Semaphore Bulkhead: limits concurrent calls using a semaphore. Simpler, same thread. Good for fast, synchronous calls. maxConcurrentCalls configuration. ThreadPool Bulkhead: runs calls in a dedicated thread pool. Different thread for each downstream. Better isolation — if pool is exhausted, caller thread returns immediately. Requires async programming model (CompletableFuture).

Q6. How does a fallback method work in Resilience4j?

If any wrapped exception reaches the boundary (including CallNotPermittedException from circuit breaker, BulkheadFullException, TimeoutException), Resilience4j calls the fallback method. Fallback must have same parameters as the primary method PLUS a Throwable parameter. Return value must be same type. The fallback can return cached data, default values, or a degraded response.

Q7. What should a fallback return?

Options (in order of preference): (1) Cached result from Redis/local cache (stale but functional), (2) Default/empty response (e.g., empty product list, 'price unavailable'), (3) Degraded but useful response (e.g., 'payment service unavailable, your order is queued'), (4) Fast failure with user-friendly error (better than hanging). Never: throw the same exception or hang. Never: return null without client handling it.

Q8. What is Chaos Engineering and why is it practiced?

Intentionally injecting failures in production (or staging) to discover resilience gaps BEFORE real failures occur. Hypothesis: 'The system can tolerate 50% of instances failing'. Experiment: kill 50% of pods. Observe. Fix gaps. Netflix pioneered this with Chaos Monkey. Principle: 'Better to break things intentionally than to be surprised by unexpected failures at 3am.'

Q9. What does Chaos Monkey for Spring Boot do?

Injects assaults into Spring beans: random latency (simulating slow downstream), random exceptions (simulating errors), application kill (simulating crash). Controlled via Actuator API at runtime—no code changes needed. Run in staging/canary to verify your circuit breakers, retries, fallbacks, and timeouts work before real failures.

Q10. How does @TimeLimiter interact with @CircuitBreaker?

@TimeLimiter wraps the call with a timeout. If timeout fires: throws TimeoutException. @CircuitBreaker counts TimeoutException as a failure. If timeouts are frequent enough ($\geq$ failureRateThreshold), circuit opens. This means slow calls eventually trip the circuit breaker — protecting against slow downstream services, not just erroring ones.

Q11. What are retryExceptions vs ignoreExceptions in Resilience4j?

retryExceptions: list of exception types that trigger retry. Default: all exceptions. ignoreExceptions: exceptions that are NOT retried (treated as non-retriable). Example: IOException is retriable (transient network). BusinessValidationException is not retriable (re-sending same invalid request won't succeed). Critically: HTTP 4xx client errors should be in ignoreExceptions — don't retry bad requests.

Q12. How do you configure Resilience4j health indicators?

Set registerHealthIndicator: true in configuration. Spring Boot Actuator /actuator/health includes circuit breaker states. Each circuit breaker appears as a health component: {"paymentService": {"status": "UP", "details": {"state": "CLOSED"}}}. OPEN circuit breaker makes health DOWN — Kubernetes readiness probe can use this.

Q13. What is the difference between Hystrix (Netflix) and Resilience4j?

Hystrix: Netflix's original circuit breaker (now in maintenance mode). Thread-pool-based, HystrixCommand, complex configuration. Resilience4j: modern replacement. Lightweight, functional/decorator approach, no classpath bloat (each feature is separate module), Kotlin coroutines support, better metrics via Micrometer, active development. Spring Cloud 2021+ uses Resilience4j by default.

Q14. When should you NOT use retry?

Don't retry: (1) Non-idempotent operations (POST without idempotency key — creates duplicate records), (2) HTTP 4xx errors (bad request — won't succeed on retry), (3) Business validation failures (wrong data), (4) Circuit is open (Resilience4j skips retry when circuit is OPEN — no point). Always combine retry with idempotency design in the target service.

Q15. How do you implement exponential backoff manually in Java?

For operations outside Resilience4j: long delay = initialDelay; for (int attempt = 0; attempt < maxAttempts; attempt++) { try { return callService(); } catch (Exception e) { if (attempt < maxAttempts-1) Thread.sleep(delay + random(jitter)); delay *= multiplier; } } Or use Spring Retry: @EnableRetry + @Retryable(maxAttempts=3, backoff=@Backoff(delay=1000, multiplier=2)).

Q16. What is Rate Limiting and when should you use it?

Rate Limiter restricts calls per time window. Two uses: (1) Protect downstream: limit your outgoing calls so you don't overwhelm a slow external API (their quota). (2) Protect yourself: limit incoming requests from clients (API gateway level). Resilience4j RateLimiter: limitForPeriod=10, limitRefreshPeriod=1s = max 10 calls/sec. Throws RequestNotPermitted when exceeded — handle with fallback.

Q17. How does Resilience4j integrate with Spring AOP?

Annotations (@CircuitBreaker, @Retry, etc.) are processed by Spring AOP. Each annotation wraps the method call in the corresponding Resilience4j decorator. Limitation: AOP only works for Spring-managed

beans calling each other via Spring proxies. Self-invocation (calling annotated method on same object) bypasses AOP — use programmatic API instead.

Q18. What is the programmatic Resilience4j API?

```
CircuitBreaker cb = CircuitBreakerRegistry.ofDefaults().circuitBreaker('name');
Supplier decorated = CircuitBreaker.decorateSupplier(cb, () -> externalService.call());
Try result = Try.ofSupplier(decorated).recover(ex -> 'fallback');
```

Use when: self-invocation, non-Spring code, dynamic circuit breaker names. More flexible but more verbose than annotations.

Q19. How do you measure resilience pattern effectiveness?

Metrics per circuit breaker: calls successful/failed/not permitted (open), slow call rate, failure rate. Metrics per retry: retry attempts distribution, successful retries. Metrics per bulkhead: concurrent calls, rejected calls (pool full). All exposed via Micrometer → Prometheus → Grafana. Alert: circuit breaker opens (indicates downstream degradation).

Q20. What is the 'thundering herd' problem and how does bulkhead help?

Thundering herd: many clients simultaneously retry or reconnect after a failure. Overwhelms the service when it recovers. Bulkhead limits concurrent calls per downstream — only N calls allowed at once. Combined with jitter in retries: clients stagger their attempts. Circuit breaker also helps: HALF-OPEN allows only permittedCallsInHalfOpenState probe calls.

Q21. Describe a real-world circuit breaker configuration for a payment service.

failureRateThreshold: 50 (open if >50% fail). slowCallDurationThreshold: 3s (payment should respond in <3s). slowCallRateThreshold: 60 (open if >60% are slow). waitDurationInOpenState: 30s (wait before probing). permittedCallsInHalfOpenState: 3 (3 probe calls). slidingWindowSize: 20 (evaluate last 20 calls). minimumNumberOfCalls: 10 (need 10 calls before evaluating).

Q22. What is 'bulkhead by tenant' in a multi-tenant system?

Separate thread pools (or semaphore limits) per tenant. If one tenant makes excessive calls or their downstream is slow, it doesn't exhaust threads for other tenants. Implemented by dynamically creating Resilience4j Bulkhead instances per tenant ID. Combined with per-tenant rate limiting to prevent noisy neighbor problems.

Q23. How do you handle fallback for a caching scenario?

```
@CircuitBreaker(name='productService', fallbackMethod='getFromCache')
public Product getProduct(String id) {
    return productClient.get(id);
}
public Product getFromCache(String id, Exception e) {
    Product cached = redisCache.get(id);
    if (cached != null) return cached;
    log.warn('Cache miss for {}, returning default', id);
    return Product.defaultProduct(id);
}
```

Q24. What is Resilience4j's interaction with Spring Cloud Gateway?

Spring Cloud Gateway supports Resilience4j circuit breaker filter out of the box. Applying circuit breaker at the gateway protects all downstream microservices without requiring each service to implement their own client-side circuit breakers. Configuration: `spring.cloud.gateway.routes[0].filters[0]=CircuitBreaker=name=myCircuit,fallbackUri=/fallback`

Q25. What is adaptive retry and when is it useful?

Standard retry: fixed number of attempts. Adaptive retry: adjusts based on system state. If circuit breaker is OPEN: skip retries (useless). If system under load: reduce retry attempts. Resilience4j doesn't have built-in adaptive retry, but combining @Retry with @CircuitBreaker achieves this — when circuit opens, @Retry is bypassed automatically.

Q26. How do you test circuit breaker behavior?

Resilience4j test: use `CircuitBreakerRegistry.ofDefaults()` in tests. Force OPEN state: `circuitBreaker.transitionToOpenState()`. Verify fallback is called. Force CLOSED: `circuitBreaker.transitionToClosedState()`. Verify primary method called. Integration test (Testcontainers): stop downstream container, verify circuit opens, verify fallback returns, restart container, verify recovery.

Q27. What is the difference between timeout at the HTTP client level vs TimeLimiter?

HTTP client timeout (`RestTemplate.setConnectTimeout`, `WebClient.timeout()`): set on the connection/read, throws `SocketTimeoutException`. `@TimeLimiter`: wraps the entire async operation including any processing after HTTP response arrives. Both should be set. Combine: HTTP client timeout = fast network failure. `@TimeLimiter` = overall operation timeout including processing.

Q28. What is Resilience4j's bulkhead vs Java Semaphore?

Java Semaphore: low-level synchronization primitive. Manual acquire/release. No metrics, no fallback integration, no Spring AOP support. Resilience4j Bulkhead: built on Semaphore internally but adds: Micrometer metrics, Spring AOP annotation support, fallback integration, configurable wait timeout, events/listeners for monitoring. Use Resilience4j Bulkhead for production code.

Q29. How should you tune circuit breaker thresholds?

Start by measuring baseline: what is the normal failure rate and latency of the downstream? Set `failureRateThreshold` above baseline (e.g., if normal is 1%, set threshold at 20%). Set `slowCallDurationThreshold` at P99 latency + buffer. Set `waitDurationInOpenState` long enough for downstream to recover (30s-60s typical). Monitor in staging with real load. Adjust based on alert sensitivity vs false-positive rate.

Q30. What is an idempotency key and why is it required for retries?

A client-generated unique ID included in a request. Server uses it to detect duplicate requests. If request is retried (network error made client think it failed), server returns cached result. Without idempotency key: POST /payments retried = double charge. With key: POST /payments + Idempotency-Key: uuid123. Second attempt returns first result. Required for any non-idempotent operation that uses retry.

Q31. What is the 'fail fast' principle in resilience?

When a downstream is known to be failing (circuit OPEN), return error immediately without making the call. Benefits: (1) Caller gets response immediately (no waiting for timeout), (2) Downstream gets no additional load (chance to recover), (3) System resources freed (no blocked threads waiting for timeouts). Fail fast is better than slow fail in microservices.

Q32. Describe the Resilience4j event listeners for monitoring.

`CircuitBreaker.EventPublisher`: `onSuccess()`, `onError()`, `onStateTransition()`, `onSlowCall()`. Subscribe programmatically: `cb.getEventPublisher().onStateTransition(event -> log.warn('Circuit changed to {}', event.getStateTransition()))`. Spring Boot auto-exposes these as Micrometer metrics (`resilience4j.circuitbreaker.calls`, `.state`, `.failure.rate`). Use Grafana to visualize state transitions.

Q33. What is the 'hedging' resilience pattern?

Send the same request to multiple replicas simultaneously. Return the first successful response. Cancel remaining. Reduces tail latency (P99) at cost of extra load. Not directly supported by Resilience4j — implement using `CompletableFuture.anyOf()`. Useful for: read-only operations, real-time systems where latency matters more than resource usage.

Q34. How does Resilience4j work with WebFlux (reactive)?

Resilience4j 2.x supports Project Reactor natively. Operators: `CircuitBreakerOperator.of(cb)`, `RetryOperator.of(retry)`. `Mono result = Mono.fromCallable(() -> externalService.call()) .transformDeferred(CircuitBreakerOperator.of(circuitBreaker)) .transformDeferred(RetryOperator.of(retry)) .onErrorReturn('fallback');` Non-blocking — resilience patterns applied in reactive pipeline.

Q35. What is Chaos Monkey Assault mode and how do you use it safely?

Assault modes: LATENCY (add random delay), EXCEPTION (throw random exception), APP_KILLER (kill application), MEMORY_ASSAULT (exhaust heap). Safe use: enable in staging/canary only. Start with LATENCY. Verify circuit breakers trip, fallbacks work, alerts fire. Run A/B: service with chaos monkey on one instance, normal on others. Never enable APP_KILLER in production without proper runbooks.

Q36. What are the best practices for resilience in Spring Boot microservices?

(1) Apply circuit breaker on ALL external calls (HTTP, Kafka producer, Redis, DB). (2) Always have a fallback — never let circuit open without a graceful degradation. (3) Add jitter to all retries. (4) Separate thread pools per downstream with bulkhead. (5) Set timeouts on every call — never rely on default infinite timeout. (6) Test with Chaos Monkey before going live. (7) Monitor circuit breaker state in Grafana — alert on OPEN state.

Q37. What is the relationship between circuit breaker and health checks?

Spring Boot Actuator health endpoint can reflect circuit breaker state. `registerHealthIndicator: true` in Resilience4j config. If circuit breaker is OPEN, service health = DOWN. Kubernetes readiness probe fails → service removed from load balancer. This prevents routing traffic to a service that cannot serve requests due to downstream failure. Liveness probe remains UP (service is alive, just degraded).

Q38. How do you handle resilience for database connections?

Connection pool (HikariCP): `connectionTimeout`, `idleTimeout`, `maxLifetime`. Circuit breaker at the repository layer: wrap DB calls with `CircuitBreaker`. If DB is down, circuit opens, fallback returns cached data. Health check: Spring Boot `DataSource` health indicator shows DB connectivity. Retry for transient connection failures (connection reset). Never retry schema/data errors — those are non-retriable.